Pune Vidyarthi Griha's College of Engineering, Technology and Management, Pune- 411009.

(Affiliated to Savitribai Phule Pune University)



Mini Project Report On

"UI Text Accessibility Reviewer"

(Natural Language Processing Mini Project)

By

Haripujan Prasad: 2063

Soham Muley: 2065

Ritika Paradkar: 2066

Under The Guidance of: **Prof. Mrs. K. P. Goswami**

Artificial Intelligence and Data Science Department

Academic Year: 2026-2027

Date of Submission:

INDEX

Sr.No.	Topic	Page No.
1.	Aim	3
2.	Problem Statement	3
3.	Objectives	4
4.	Libraries/Packages Used	5
5.	Introduction	6
6.	Theory	7
7.	Methodology	9
8.	Architecture Diagram	12
9.	Component Diagram	14
10.	Dataset Details	16
11.	Conclusion	18
12.	Output	19

1. Aim

The primary aim of this mini project is to develop a web application, the **UI Text Accessibility Reviewer**, that evaluates the text found in a user interface — button labels, links, form labels, headings and system messages — for **clarity, accessibility, reading level and consistency**, and then recommends concrete changes. The application accepts three kinds of input: the address of a website, a screenshot of a website or application, or plain text pasted by the user. It is built with the Python programming language and Natural Language Processing (NLP) techniques.

2. Problem Statement

Every website and application communicates with its users through small pieces of text: the word on a button, the name of a form field, the sentence in an error message. When this text is vague ("Submit", "Click here"), technical ("Error 0x80070057: invalid parameter"), written in a difficult style, or used inconsistently ("Log in" on one screen and "Sign in" on the next), users hesitate, make mistakes or give up. For people who use screen readers, have low vision, dyslexia or limited literacy, poor interface text can make a product impossible to use.

Reviewing interface text by hand is slow and subjective. Automated accessibility checkers largely concentrate on page structure and code, and they say little about the quality of the words themselves. A tool is therefore required that can read the text of an interface, judge it against accepted guidelines, and explain what to change.

This project addresses the problem statement **"Evaluate UI labels, buttons and messages for clarity, accessibility, reading level and consistency"**. The system reads the text of a website (including pages built with JavaScript), of a screenshot, or of pasted content, applies a transparent set of NLP-based rules, measures colour contrast and text size in screenshots, and produces a scored report with prioritised recommendations and suggested rewrites.

3. Objectives

The specific objectives of this project are as follows:

I. To study guidelines for clear and accessible interface text (the Web Content Accessibility Guidelines and plain-language practice) and convert them into measurable rules.

II. To acquire interface text from three sources: the HTML of a website (including pages rendered by JavaScript in a headless browser), a screenshot read with optical character recognition (OCR), and text typed or uploaded by the user.

III. To classify each piece of text as a button, link, field label, heading, message, image description or paragraph, because the rules that apply to each differ.

IV. To check **clarity** using word lists and patterns for jargon, vague actions, abbreviations, passive voice, double negatives, error codes and error messages that give no way to fix the problem.

V. To check **accessibility** by detecting generic link text, colour- or position-only instructions, device-specific wording, ALL CAPS, emoji, symbol-only text and page-level problems such as images without alt text and form fields without labels.

VI. To measure **reading level** with the Flesch Reading Ease and Flesch–Kincaid Grade formulas using a built-in syllable counter, and to check **consistency** of terms, capitalisation and punctuation across all strings.

VII. To measure **colour contrast and text size** from screenshots and compare them with WCAG recommendations.

VIII. To calculate scores, list the most serious problems first, propose automatic rewrites, and let the user export the report; and to verify the system with automated tests and demonstration pages.

4. Libraries / Packages Used

I. Flask:

- Lightweight Python web framework that serves the web pages and the REST API (/api/analyze, /api/scan, /api/screenshot).
- Handles JSON requests, file uploads and the built-in demonstration pages.

II. re (Regular Expressions):

- Used for pattern matching: passive voice, double negatives, error codes, colour- or position-based instructions and input prefixes such as “Button label:”.

III. html.parser (HTMLParser):

- Standard-library parser that walks through a page’s HTML and collects buttons, links, labels, headings, messages, alt text and paragraphs.

IV. urllib, socket, ipaddress:

- Download web pages safely: every redirect is checked and addresses that point to internal networks are refused.

V. Playwright (headless Chromium):

- Opens a page in a hidden browser so that its JavaScript runs, which is needed for modern single-page applications.
- Every network request made by the page is checked before it is allowed.

VI. RapidOCR with ONNX Runtime:

- Text detection and recognition for screenshots; returns each piece of text with its position on the image.

VII. Pillow and NumPy:

- Load images and calculate colours, contrast ratios and text height from pixel data.

VIII. HTML5, CSS3 and JavaScript:

- Front end of the application: tabs, upload area, results dashboard, filters and exports, written without any framework.

IX. pytest:

- Runs the 21 automated tests that check the rules, the website scanner, the screenshot reviewer and the API.

5. Introduction

Interface text is part of the design of every digital product, yet it is often written quickly and reviewed rarely. A button that says “OK” does not tell the user what will happen. A link that says “Click here” is meaningless to a person who navigates by listening to a list of links. An error message such as “Invalid parameter” explains nothing and offers no way forward. Such problems affect everyone, but they affect people with disabilities most.

This project builds a web application, the UI Text Accessibility Reviewer, that acts like a proofreader for interface text. The user enters a website address, uploads a screenshot, or pastes text. The system extracts every piece of interface text, works out what kind of element it is, and reviews it against four dimensions: clarity, accessibility, reading level and consistency. For each problem it explains why it matters and what to change, and for many strings it proposes a rewritten version.

The reviewer uses a transparent, rule-based NLP approach instead of a black-box model. Each finding can be traced to a named rule, such as a word list, a pattern or a formula, so that a designer or writer can understand and trust the advice. The system works offline, needs no training data and is fast. For websites it can read pages that are built with JavaScript by running them in a headless browser, and for screenshots it uses OCR and image measurement to find low colour contrast and very small text.

The results are presented as a dashboard with an overall score, a list of the most serious problems, and a collapsible list of every string. The report can be exported as CSV or JSON or printed to PDF. The project demonstrates practical use of natural language processing, text analytics, web scraping and computer vision in the field of digital accessibility.

6. Theory

Why interface text matters. Interface text (also called microcopy) guides users through tasks. Good text is short, specific, uses everyday words and describes what will happen. The Web Content Accessibility Guidelines (WCAG) from the World Wide Web Consortium set the accepted standard for accessible content. The success criteria most relevant to this project are listed below.

WCAG criterion	Requirement (summary)	Checked by this project
1.1.1 Non-text content	Images need a text alternative	Images without alt text; alt text that is a file name
1.3.1 / 3.3.2	Form fields have labels or instructions	Fields without a label; placeholder used as label
1.3.3 Sensory characteristics	Instructions must not rely only on colour, shape or position	“the red button on the left”
1.4.3 Contrast (minimum)	Text contrast of at least 4.5:1 (3:1 for large text)	Contrast measured in screenshots
1.4.4 Resize text	Text can be enlarged	Pinch-zoom disabled; very small text
2.4.4 Link purpose	Link text describes its destination	“Click here”, “Read more”
3.1.1 Language of page	The page language is declared	Missing lang attribute
3.3.1 / 3.3.3 Errors	Errors are identified and a correction is suggested	Error messages with no way to fix the problem

Types of text analysis

Rule-based (lexicon and pattern) analysis:

Word lists and regular-expression patterns identify known problems, for example formal words such as “utilize” or “authenticate”, vague button words such as “Submit” or “OK”, or constructions such as “is initialized by”. This approach is transparent, needs no training data and gives exact explanations.

Statistical and machine-learning analysis:

Classifiers or language models can judge tone and fluency but need large labelled datasets, are harder to explain and may behave unpredictably. For an accessibility review where every recommendation must be justified, a rule-based method was chosen for this project, with the design allowing models to be added later.

Readability formulas

Reading level is estimated with two classic formulas, where words per sentence and syllables per word are measured on the text:

- **Flesch Reading Ease** = 206.835 − 1.015 × (words ÷ sentences) − 84.6 × (syllables ÷ words). Higher scores mean easier text; 60–70 is considered plain English.

- **Flesch–Kincaid Grade Level** = 0.39 × (words ÷ sentences) + 11.8 × (syllables ÷ words) − 15.59. The result approximates a school grade; plain-language guidance recommends about grade 8 or lower.

The formulas are unreliable on very short strings, so the project applies them only to text of six or more words. Syllables are counted with a vowel-group heuristic.

Colour contrast

WCAG defines contrast as (L1 + 0.05) ÷ (L2 + 0.05), where L1 and L2 are the relative luminances of the lighter and darker colour. Relative luminance is 0.2126 R + 0.7152 G + 0.0722 B after each sRGB channel is linearised. A ratio of 4.5:1 is required for normal text and 3:1 for large text.

Optical character recognition (OCR)

OCR converts text in an image into characters. Modern OCR has two stages: a detection network finds the regions that contain text and returns their bounding boxes, and a recognition network reads the characters in each region. Because the bounding boxes give the position and size of every piece of text, they can also be used to measure its height and the colours around it.

7. Methodology

The methodology for building the UI Text Accessibility Reviewer is as follows:

1. Input Acquisition:

- **Website address:** the page is downloaded over HTTP(S). Redirects are followed one by one and each address is checked; internal network addresses are refused. If the page has little text in its HTML, or the user selects the option, it is opened in a headless browser so that JavaScript-built content is included.
- **Screenshot:** an image up to 10 MB is read with OCR.
- **Pasted text:** one string per line, optionally with a type such as “button | Save” or “Error message: …”.

2. Extraction and Classification:

- The HTML parser collects buttons, links, field labels and placeholders, headings, alert and error messages, image alt text and paragraphs, and remembers where each came from.
- For screenshots, wrapped lines are merged into paragraphs and each text area is classified (for example a short text on a filled background is treated as a button).

3. Pre-processing:

- Text is split into words and sentences, counted, and syllables are estimated for the readability formulas.

4. Clarity and Accessibility Rules:

- Each string is checked by rules that depend on its type. Examples: vague button labels, jargon, abbreviations, error codes, passive voice, double negatives, error messages without a remedy, generic links, sensory instructions, ALL CAPS, emoji and symbol-only text.

5. Reading Level:

- Flesch Reading Ease and Flesch–Kincaid Grade are calculated for strings of six or more words; a grade above 8 raises a medium issue and above 12 a high issue.

6. Consistency Across Strings:

- All strings are compared with each other: different words for the same idea (Log in / Sign in / Login), mixed capitalisation styles, mixed end punctuation and near-duplicates.

7. Page-level and Visual Checks:

- For websites: missing page language or title, images without alt text, unlabeled form fields, unnamed buttons and links, duplicate link text, heading-level problems and disabled zoom.
- For screenshots: colour contrast against the local background and estimated text size.

8. Scoring:

- Every string starts at 100 and loses 22, 12 or 5 points for each high, medium or low issue. The overall score is the average of the string scores; for websites and screenshots it is blended 50:50 with a page-level score calculated in the same way.

9. Recommendation:

- Each issue includes a suggested fix. Safe automatic rewrites are offered, for example “CANCEL” → “Cancel” or “prior to” → “before”, and the most serious problems are listed first.

10. User Interface and Export:

- A tabbed web interface accepts the input and shows a score card, a “Fix these first” list, annotated screenshots and a collapsible list of all strings with filters. Results can be exported as CSV or JSON or printed to PDF, and the text size can be enlarged.

Algorithm

1. Start

2. Receive the input

- Website address, screenshot or pasted text.

3. Acquire the content

- Website: fetch the HTML; if the text is too thin or JavaScript is requested, render the page in a headless browser.
- Screenshot: run OCR to obtain text and bounding boxes.

4. Extract and classify the text

- Build a list of (text, element type, location) entries.

5. Analyse each entry

- Run the clarity and accessibility rules for its element type.
- Compute reading ease and grade if it has six or more words.
- For screenshots, measure contrast and text size.

6. Compare all entries

- Find inconsistent terms, capitalisation and punctuation.

7. Run page-level checks

- Check language, title, alt text, labels, names of buttons and links, headings and zoom.

8. Score

- Deduct 22 / 12 / 5 points for each high / medium / low issue; average and blend the scores.

9. Recommend

- Attach a suggestion to each issue, generate rewrites, sort by severity.

10. Display and export

- Show the dashboard; offer CSV, JSON and print output.

11. End

Examples of rules applied

Dimension	Rule	Example input	Recommendation
Clarity	Vague button	Submit	Use verb + object, e.g. “Send message”
Clarity	Error code in message	Error 0x80070057: invalid parameter	Explain in words and add the next step
Accessibility	Generic link text	Click here	Describe the destination
Accessibility	Sensory instruction	Press the red button on the left	Refer to the button by name
Accessibility	ALL CAPS	CANCEL	Use sentence case: “Cancel”
Reading level	Grade above 8	…the payload was malformed and the credentials were not authenticated…	Shorter sentences, everyday words
Consistency	Term variants	Log in / Sign in / Login	Choose one term
Visual	Low contrast	Light grey text on white (1.9:1)	Use a darker colour (4.5:1 or more)

Severity levels and scoring

Severity	Points deducted	Meaning and typical example
High	22	Blocks or seriously confuses users, or breaks an accessibility requirement. Example: “Click here” as link text; an error message with no way to fix the problem; an image without alt text.
Medium	12	Makes the text harder to understand or use. Example: jargon such as “credentials”; ALL CAPS; a reading grade above 8.
Low	5	A polish or consistency issue. Example: a wordy phrase; passive voice; a different capitalisation style from the other buttons.

Worked example. The message “Error 0x80070057: invalid parameter” has two high issues (an error code and no way to fix the problem) and one medium issue (technical wording). Its score is 100 − (22 + 22 + 12) = **44**. The overall score is the average of all string scores. For a website the average is then blended equally with the page-level score; the badly written demo website has a text score of 79 and a page-level score of 0, so its overall score is (79 + 0) ÷ 2 = **40**.

Scores are grouped into four bands: 90 and above is Excellent, 75 to 89 is Good, 55 to 74 Needs work, and below 55 is Poor.

8. Architecture Diagram

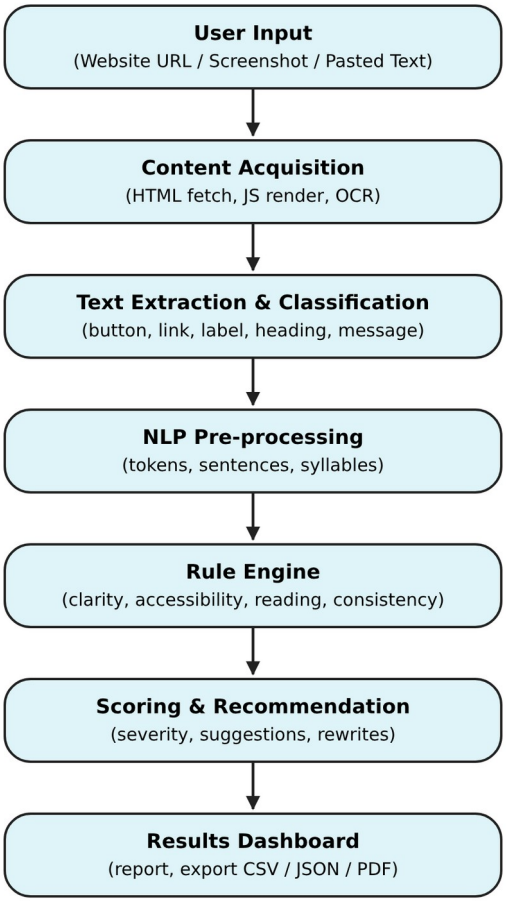


Figure 8.1: Architecture of the UI Text Accessibility Reviewer

The architecture of the system includes the following modules:

1. **Input Layer**
   - Accepts a website address, a screenshot image or pasted text from the web interface.
2. **Content Acquisition Module**
   - Downloads pages safely, renders JavaScript pages in a headless browser, or reads screenshots with OCR.
3. **Extraction and Classification Module**
   - Turns a page or an image into a list of interface strings with their element types and locations.
4. **NLP Pre-processing Module**
   - Tokenises text into words and sentences and estimates syllables.
5. **Rule Engine**
   - Applies clarity, accessibility, reading-level and consistency rules, page-level checks and, for screenshots, contrast and size measurements.
6. **Scoring and Recommendation Module**
   - Calculates scores, ranks problems by severity, and prepares suggestions and rewrites.
7. **Presentation Layer**
   - Shows the results dashboard and provides filters and CSV, JSON and print exports.

9. Component Diagram

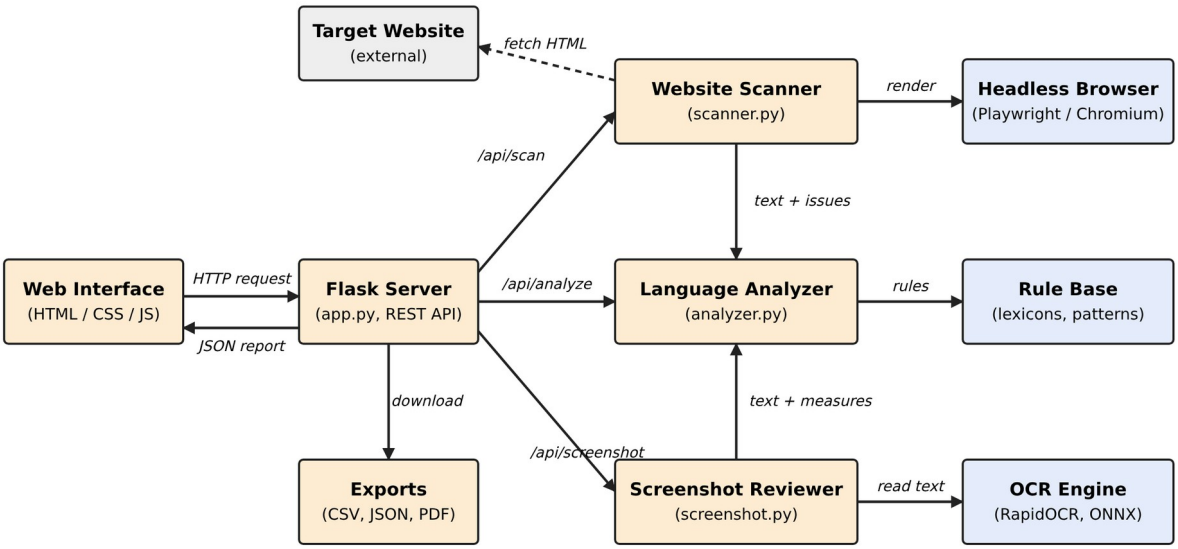


Figure 9.1: Component diagram

1. Web Interface (HTML / CSS / JavaScript):

- Function: where the user interacts with the system.
- Input: website address, screenshot file or pasted text.
- Output: score card, recommendations, annotated screenshot, exports.
- Communication: sends HTTP requests to the Flask server and receives JSON.

2. Flask Server (app.py):

- Function: central hub that exposes the REST API and serves the pages.
- Routes: /api/analyze (text), /api/scan (website), /api/screenshot (image) and /demo/* (built-in demonstration pages).
- Role: validates input, limits sizes, calls the right service and combines the page-level and text scores.

3. Website Scanner (scanner.py):

- Function: fetches a page, follows redirects safely and blocks internal addresses.
- Parses the HTML into interface strings and page-level findings.
- Uses the headless browser when JavaScript has to be run.

4. Headless Browser (Playwright / Chromium):

- Function: runs the page's JavaScript and returns the rendered HTML.
- Every network request is checked against the same address rules.

5. Screenshot Reviewer (screenshot.py):

- Function: reads text from an image, merges wrapped lines, classifies text areas and measures size and colour contrast.

6. OCR Engine (RapidOCR / ONNX Runtime):

- Function: detects text regions and recognises their characters.

7. Language Analyzer (analyzer.py):

- Function: core NLP engine that runs the rules, readability formulas, consistency checks, scoring and rewrites.

8. Rule Base:

- Function: lexicons (jargon, vague actions, abbreviations, synonyms) and patterns used by the analyzer.

9. Exports:

- Function: CSV and JSON downloads and print-to-PDF.

10. Dataset Details

This project is rule-based, so it does not use a training dataset. Instead it relies on a curated **rule base** (lexicons and patterns) and on test inputs prepared for demonstration and testing.

Rule base (lexicons)

Resource	Entries	Used for
Jargon / formal words with plain alternatives	34	Clarity advice
Vague action words (Submit, OK, Go …)	15	Vague button labels
Generic link phrases (click here, read more …)	8	Link purpose
Negation words	16	Double negatives
Abbreviations (pwd, msg, cfg …)	14	Unclear short forms
Synonym groups for the same concept	12 (50 terms)	Consistency of terms
Safe automatic word swaps	11	Suggested rewrites
Wordy phrases (in order to, prior to …)	7	Filler detection and rewrites

In total the system applies **47 rules**: 31 text rules, 13 page-level checks and 3 screenshot checks.

Interface elements extracted from a website

i. **button** – <button>, role=button, input type submit/button
ii. **link** – <a href> text or aria-label
iii. **label** – <label>, <legend>, placeholders, aria-label
iv. **heading** – <h1> to <h6>
v. **message** – role=alert/status and elements named error, alert, warning, success …
vi. **alt** – alt text of images
vii. **other** – paragraphs and list items of six or more words

Test inputs

The following inputs were prepared to demonstrate and test the system. They are included in the application so that it can be shown without an internet connection.

I. **Sample text set:** 19 interface strings (buttons, labels and messages) with deliberate problems.

II. **Badly written demo website:** a page with missing alt text, unlabeled fields, vague buttons, technical errors and more.

III. **Well written demo website:** a page that follows the guidelines.

IV. **JavaScript-built demo app:** a page whose content exists only after its script runs.

V. **Sample screenshot:** a mock login screen with low contrast, small text and unclear wording.

Results on the test inputs

Input	Overall score	Text score	Page checks	Strings reviewed	Issues (H / M / L)
Sample text set	82	82	–	19	5 / 16 / 9
Badly written demo website	40	79	0	28	10 / 25 / 16
Well written demo website	100	99	100	14	0 / 0 / 2
JavaScript-built demo app	60	87	34	9	4 / 2 / 1
Sample screenshot	74	76	73	12	6 / 10 / 6

The badly written website scores far lower than the well written one, which shows that the scoring separates good and poor interface text. The 21 automated tests (11 for the language rules, 7 for the website scanner and 3 for the screenshot reviewer) all pass.

11. Conclusion

The UI Text Accessibility Reviewer successfully demonstrates how natural language processing can support digital accessibility. The system reads the text of a website, a screenshot or pasted content, classifies each piece of text, and evaluates it for clarity, accessibility, reading level and consistency. Every finding is linked to a named rule and is accompanied by an explanation and a suggested fix, so that designers and writers can act on it immediately.

The approach combines text extraction, rule-based NLP, readability formulas, cross-string comparison and image measurement. Running pages in a headless browser allows modern JavaScript applications to be reviewed, and OCR together with contrast and size measurement allows a review even when a site cannot be scanned. The results dashboard prioritises the most serious problems and supports export, which makes the tool practical for real review work.

Limitations. The reviewer is rule-based, so it can only detect problems that its rules and word lists describe. OCR, contrast and size values taken from screenshots are estimates. Pages that require a login, or that block automated tools, cannot be scanned and are best reviewed through a screenshot. The reading-level formulas are designed for sentences and are applied only to strings of six or more words. The tool supports a human reviewer and does not replace testing with real users and assistive technology.

Future scope. The system can be extended to crawl several pages of a site at once, to support Hindi and Marathi interface text, to add a machine-learning model for tone and fluency, to check contrast directly in the page’s styles, to be offered as a browser extension, and to be integrated into design tools and continuous-integration pipelines so that interface text is reviewed automatically before release.

12. Output

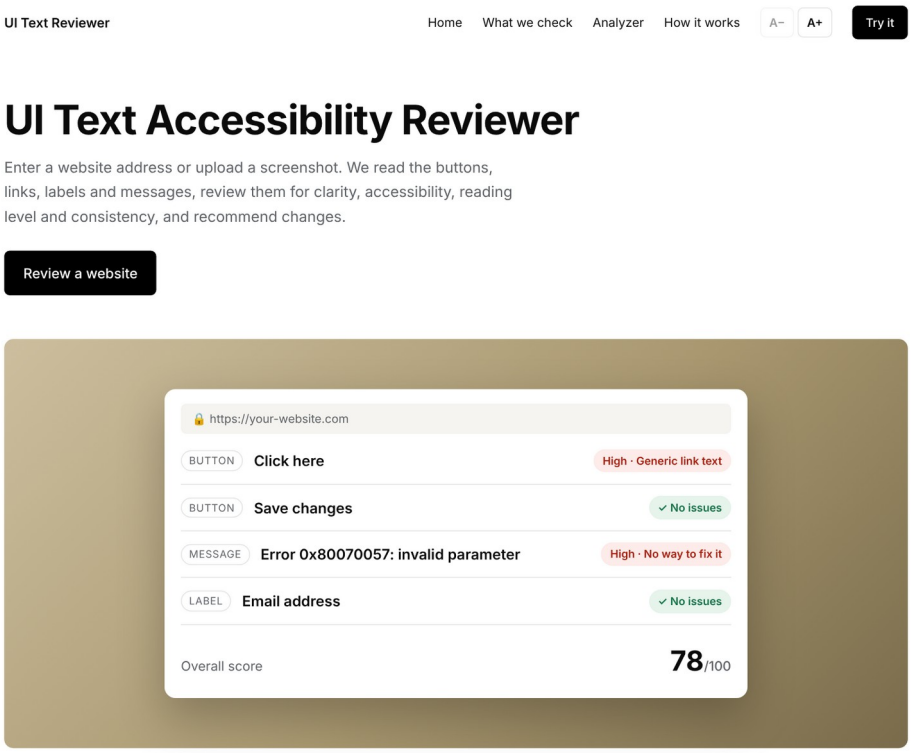


Figure 12.1: Home page of the UI Text Accessibility Reviewer

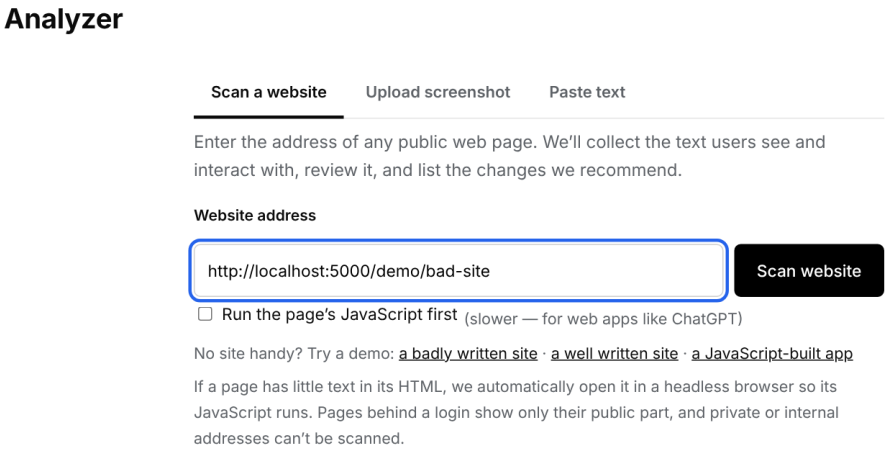


Figure 12.2: Website scan input with the built-in demonstration websites

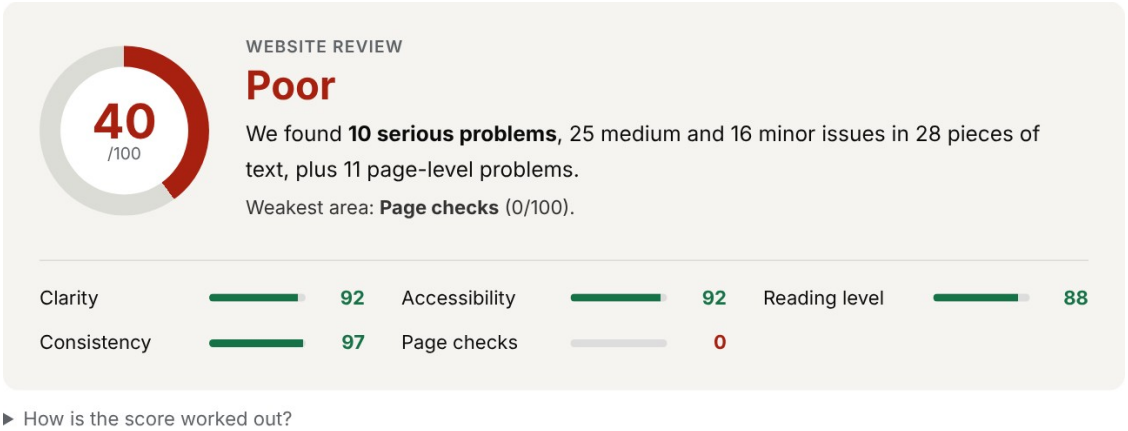


Figure 12.3: Score card for the badly written demo website (score 40, "Poor")

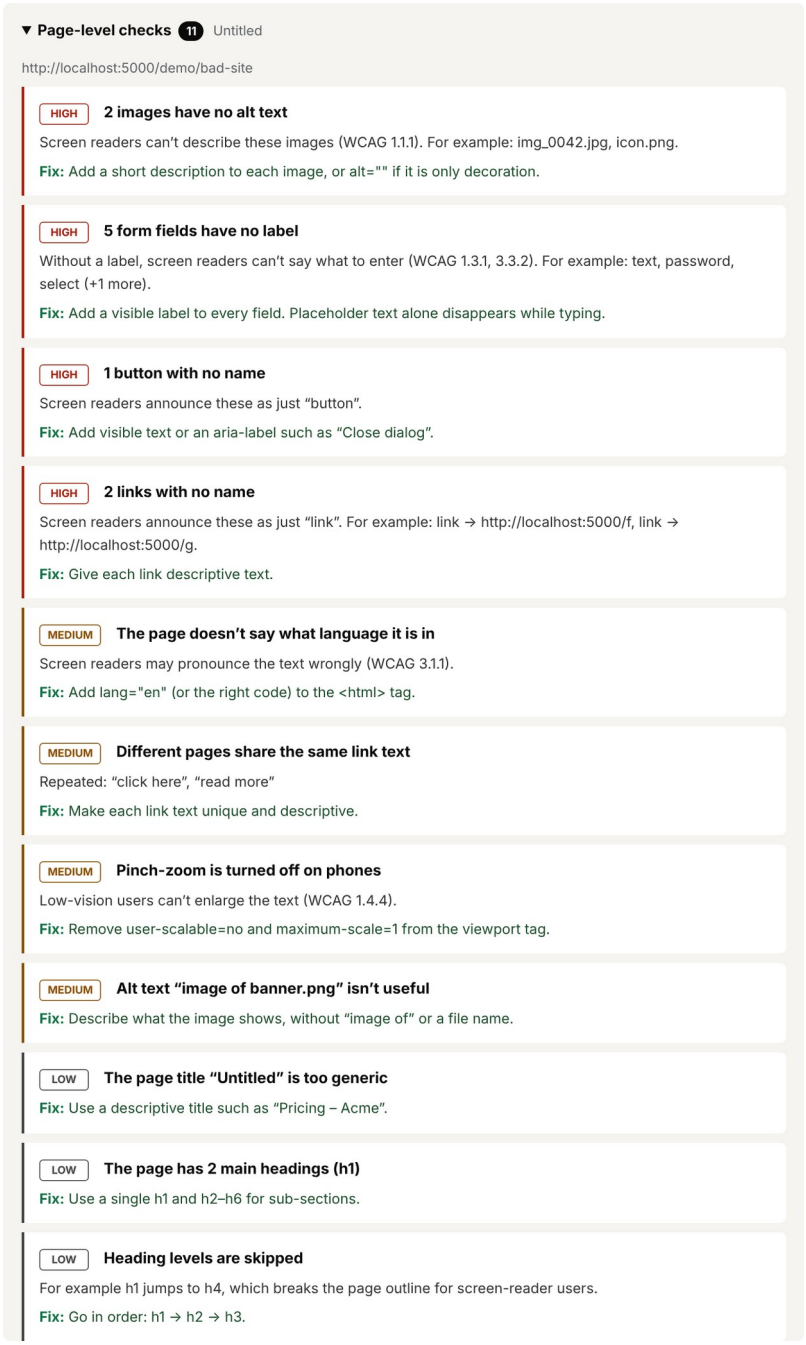


Figure 12.4: Page-level checks found on the demo website

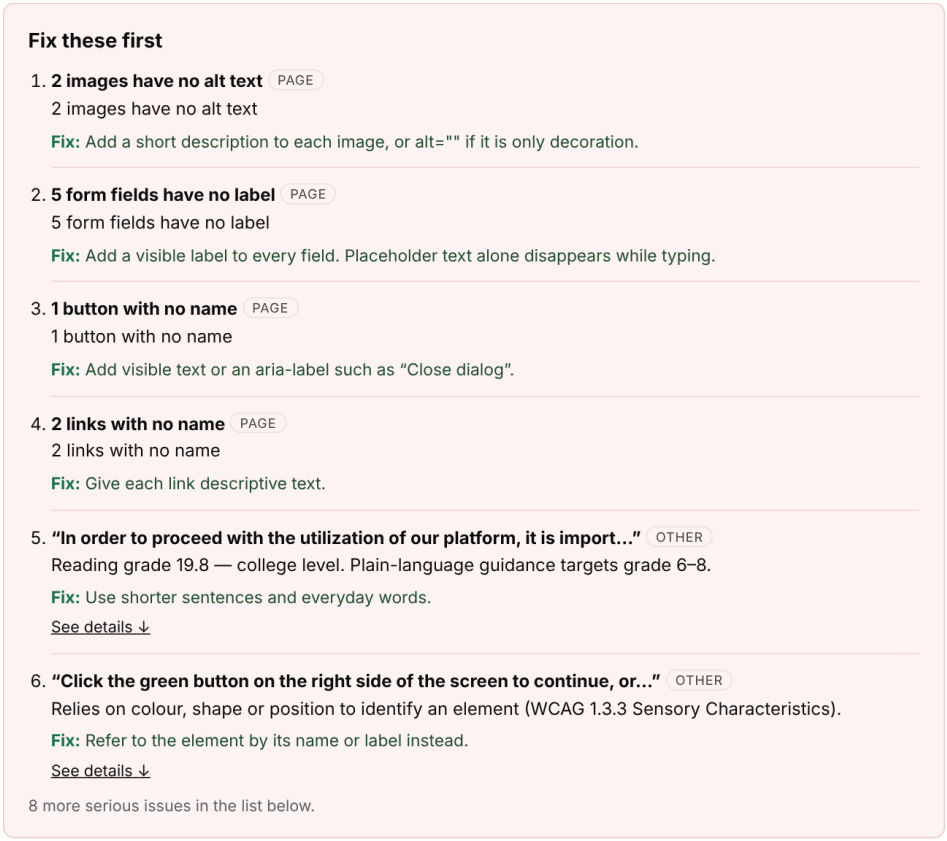


Figure 12.5: “Fix these first” list with a suggested fix for every problem

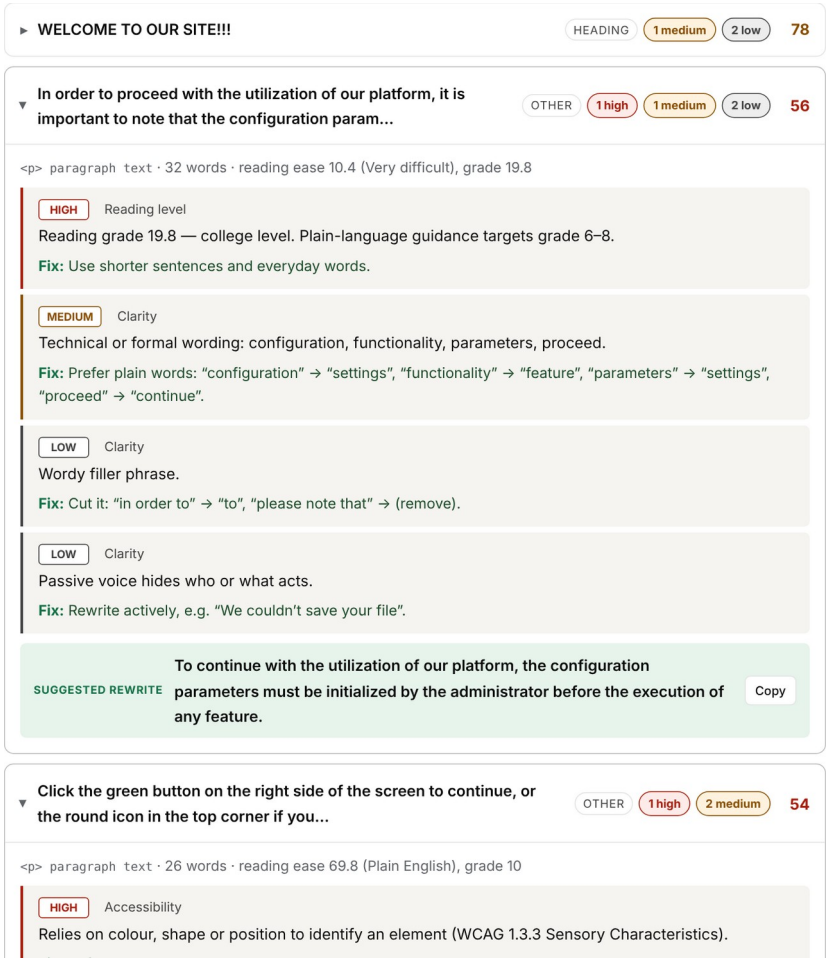


Figure 12.6: Collapsible list of every string with issues and suggested rewrites

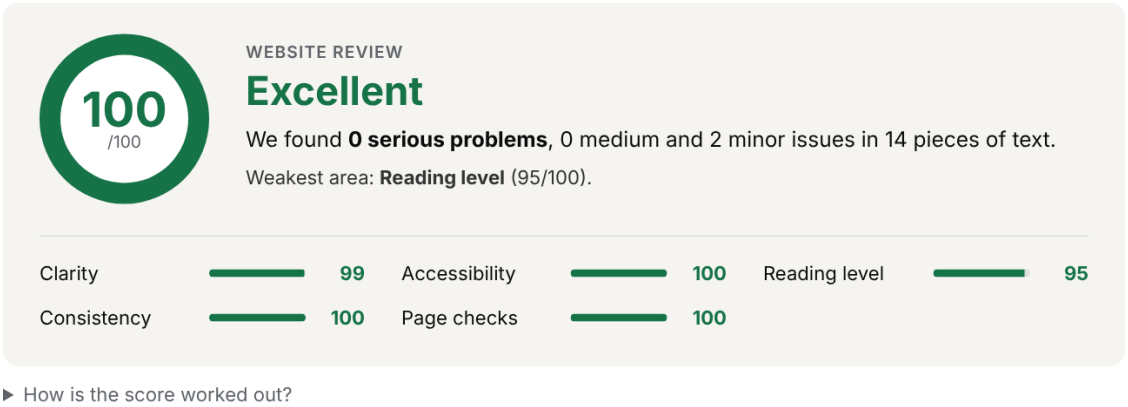


Figure 12.7: Score card for the well written demo website (score 100, "Excellent")

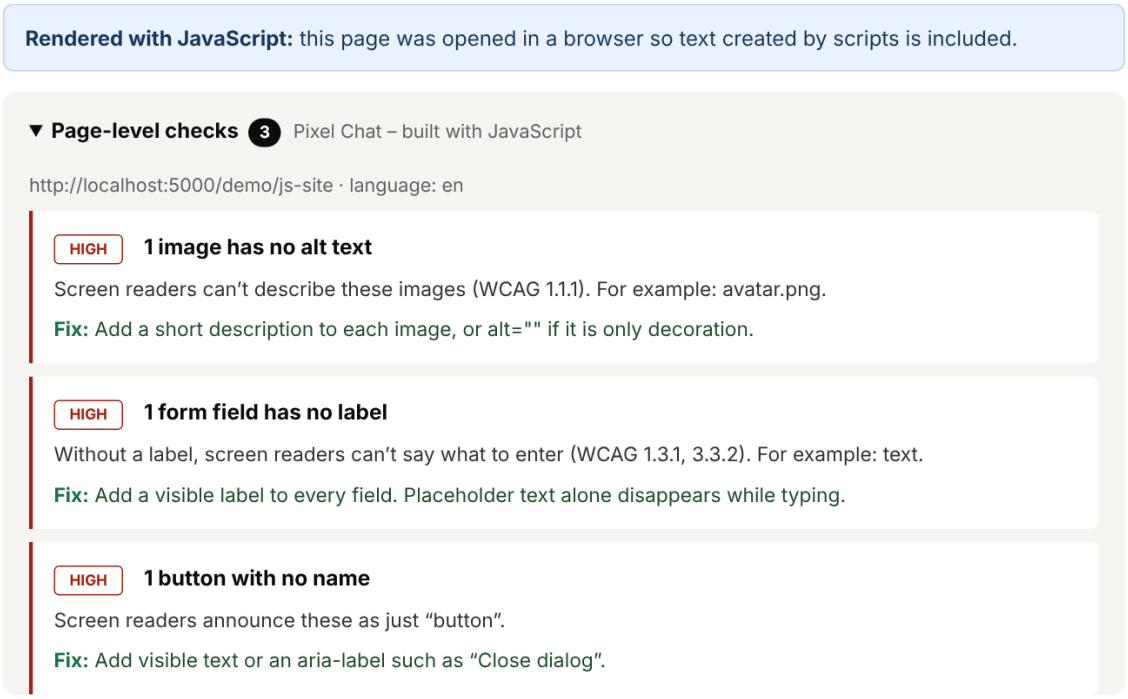


Figure 12.8: Result for the JavaScript-built demo app, read after running its script

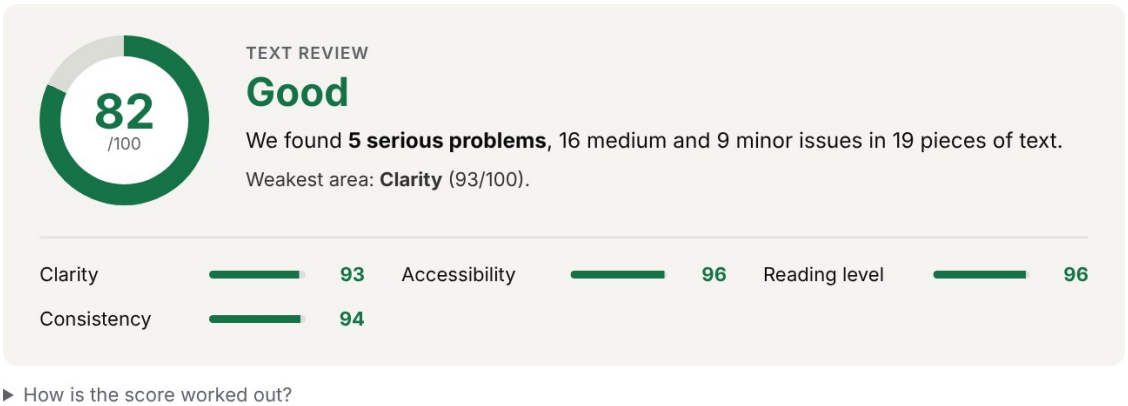


Figure 12.9: Score card for the pasted sample text

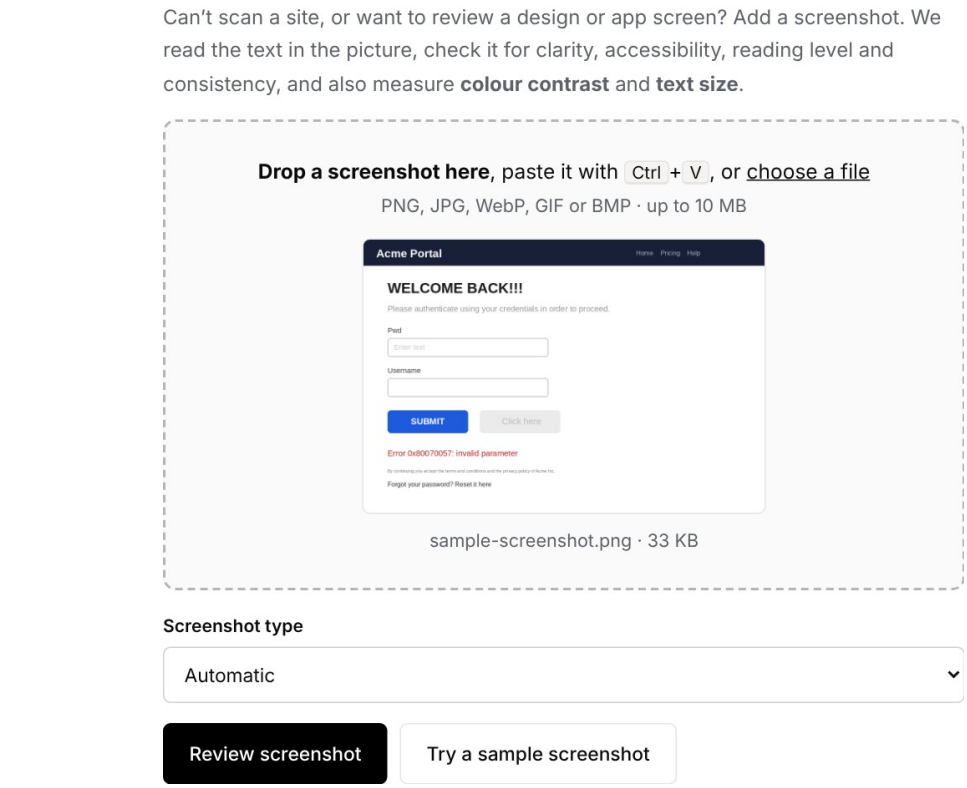


Figure 12.10: Screenshot upload tab

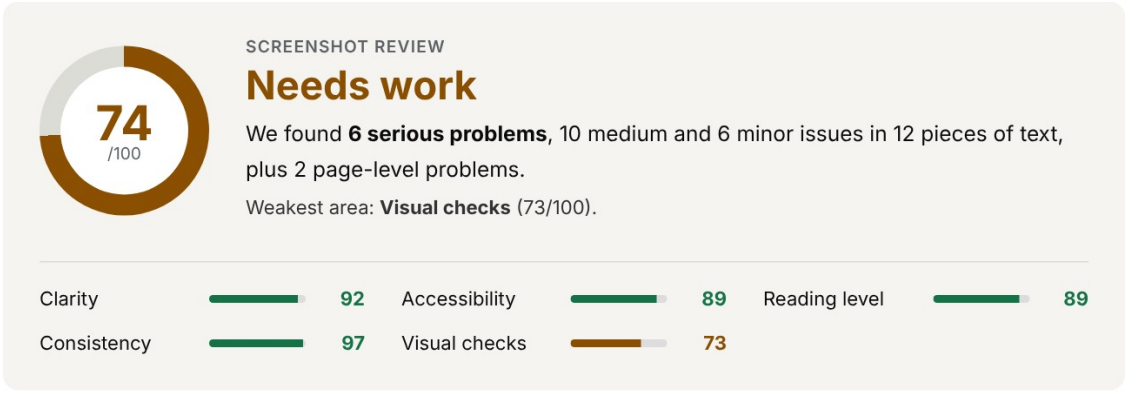


Figure 12.11: Score card for the sample screenshot

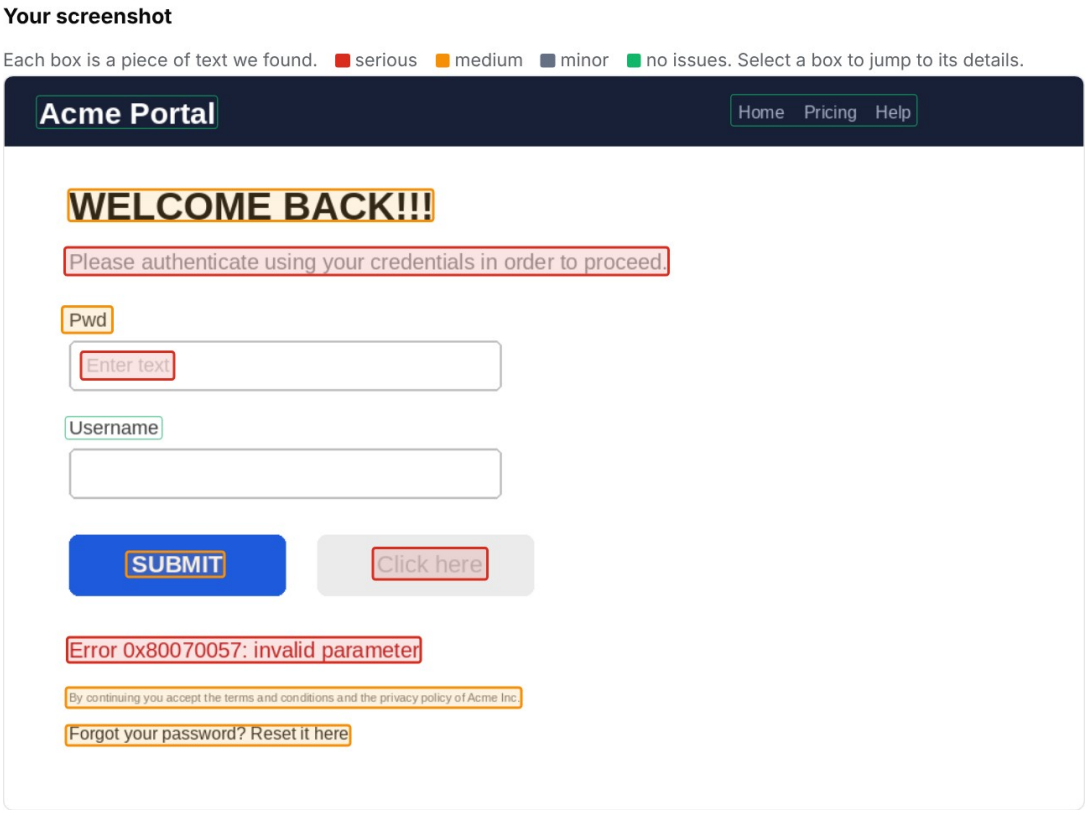


Figure 12.12: Screenshot with a box around every piece of text found (red = serious, orange = medium)

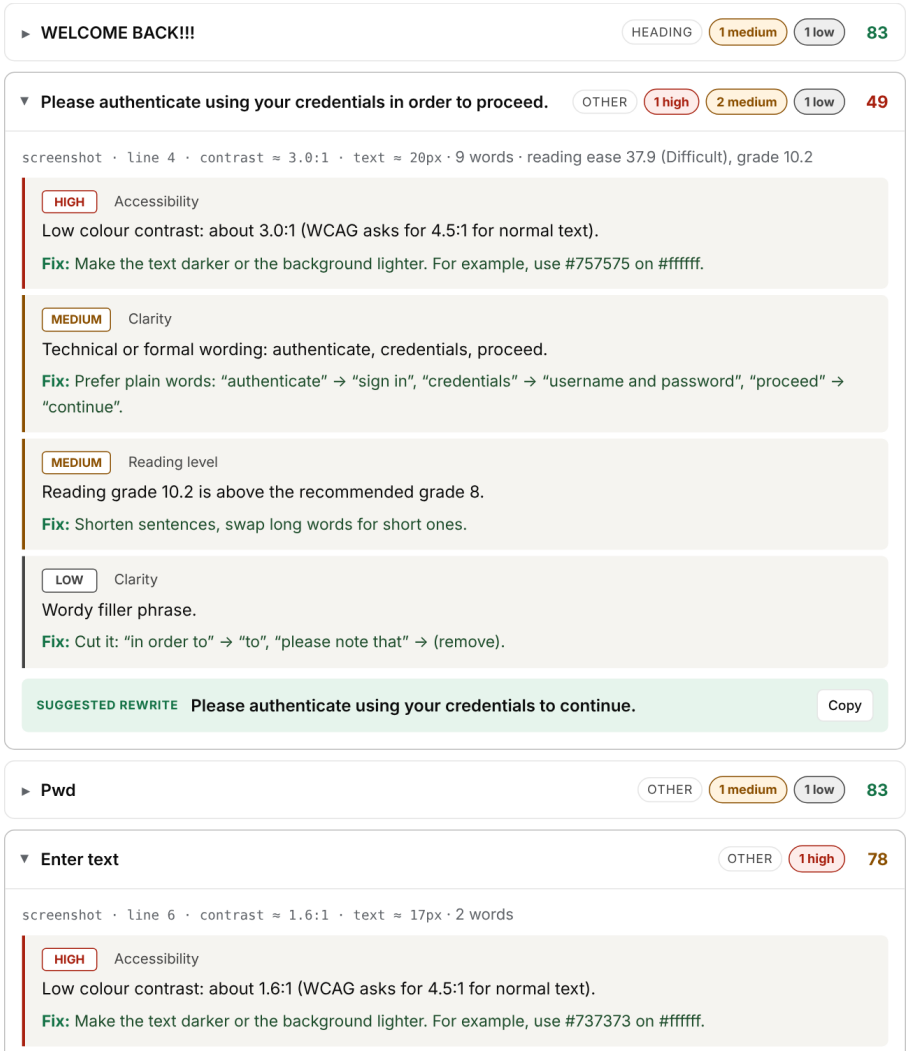


Figure 12.13: Details for the text found in the screenshot, including contrast and size findings